

HƯỚNG DẪN CHI TIẾT THỰC HIỆN BÀI TẬP LỚN

Học phần: Phân tích thiết kế hướng đối tượng (OOAD)

I. TƯ DUY TỔNG THỂ

Bài tập lớn (BTL) không phải là bài tập lập trình thuần túy. Mục tiêu là "**Tư duy mô hình hóa**". Sản phẩm cuối cùng của bạn phải thể hiện được sự chuyển dịch logic từ:

Bài toán trong thế giới thực (Bài toán nghiệp vụ) → Mô hình phân tích (Cái gì?) → Mô hình thiết kế (Như thế nào?) → Mã nguồn (Code).

Sinh viên cần tránh tư duy "code trước, vẽ sau". Mọi dòng code phải phản ánh đúng thiết kế đã được chứng minh qua biểu đồ UML và các nguyên lý thiết kế. Nên coi báo cáo BTL như một bản hợp đồng kỹ thuật giữa nhóm sinh viên và giảng viên, nơi các bạn chứng minh năng lực phân tích và thiết kế của mình.

II. QUY TRÌNH THỰC HIỆN CHI TIẾT

Bước 1: Thành lập nhóm và Lựa chọn đề tài

- **Quy mô:** Khoảng 3-5 sinh viên/nhóm hoặc theo phân công của giảng viên. Các thành viên trong nhóm nên bầu một nhóm trưởng và phân công vai trò rõ ràng cho từng người.
- **Chọn đề tài:** Ưu tiên các bài toán có nghiệp vụ rõ ràng, dữ liệu phong phú. Tránh chọn đề tài quá mơ hồ hoặc quá phức tạp về thuật toán. **Quan trọng:** Xác định rõ **phạm vi** (scope) của đề tài ngay từ đầu sao cho phù hợp với thời gian thực hiện (thường là 2-3 tuần). Ví dụ: "Quản lý thư viện" có thể chỉ dừng ở mượn/trả sách, hoặc mở rộng sang đặt trước, phạt, quản lý bản sao.
- **Khảo sát:** Thực hiện phỏng vấn (thực tế hoặc giả định) hoặc đọc tài liệu để hiểu quy trình nghiệp vụ. Ghi chép lại các vấn đề, các "nỗi đau" (pain points) của hệ thống cũ.

Bước 2: Phân tích Yêu cầu

Đây là giai đoạn tìm hiểu bài toán, xây dựng "tư điển nghiệp vụ".

1. **Xác định Tác nhân (Actors):** Là vai trò (Role), không phải con người cụ thể. Ví dụ: "Thủ thư", "Độc giả", "Quản trị viên". Tác nhân cũng có thể là một hệ thống ngoài (ví dụ: Cổng thanh toán).
2. **Xác định Ca sử dụng (Use Cases):** Dựa trên mục tiêu của tác nhân. Mỗi UC phải mang lại giá trị đo lường được.
 - *Lưu ý:* Không nên vẽ sơ đồ quá "mịn". Ví dụ: "Đăng nhập", "Nhập mật khẩu" thường không phải là UC chính, hãy gộp chúng vào UC "Xác thực người dùng".

3. Vẽ Use Case Diagram:

- Ngoài quan hệ giữa Tác nhân và UC, nên sử dụng: (1) quan hệ <<include>> cho các phần logic chung (ví dụ: "Mượn sách" include "Kiểm tra tình trạng sách") và (2) quan hệ <<extend>> cho các chức năng bổ sung tùy chọn (ví dụ: "Thanh toán" extend "Áp dụng mã giảm giá").
 - Thể hiện rõ Boundary (khung hệ thống) và mối quan hệ giữa các Actor.
4. **Đặc tả Use Case (Use Case Description):** Viết dưới dạng bảng hoặc văn bản. Bao gồm: Tên UC, Tác nhân, Mục tiêu, Tiền điều kiện, Luồng chính (Main Flow), Luồng phụ/ngoại lệ (Alternative/Exception Flow), Hậu điều kiện.
5. **Yêu cầu phi chức năng (NFRs):** Ghi rõ các ràng buộc về hiệu năng, bảo mật, môi trường triển khai, ...

Bước 3: Phân tích Đối tượng

Giai đoạn này xác định thực thể (Entity) mà không quan tâm đến UI hay CSDL.

1. **Phân tích danh từ (Noun Analysis):** Gạch chân danh từ trong đặc tả yêu cầu để tìm lớp ứng viên.
 - *Tiêu chí giữ:* Thực thể bền vững, có dữ liệu và hành vi đi kèm.
 - *Tiêu chí loại:* Chi tiết giao diện (Form, Button), chi tiết kỹ thuật (Database, Session), các hành động tức thời không có vòng đời.
2. **Conceptual Model (Class Diagram):** Chỉ thể hiện các lớp thực thể (Entity), quan hệ (Association, Aggregation, Composition, Generalization) và bội số (Multiplicity). Lưu ý:
 - Đừng vẽ thuộc tính id (khóa chính) vào mô hình khái niệm – Nó là chi tiết cài đặt.
 - Sử dụng Lớp kết hợp (Association Class) khi mối quan hệ giữa hai lớp cần lưu trữ dữ liệu (ví dụ: Member và Book có lớp Loan để lưu borrowDate, dueDate).
3. **Thẻ CRC (Class-Responsibility-Collaborator):** Đây là một công cụ quan trọng để thực hành Brainstorming.
 - Phân bổ "Trách nhiệm" (Responsibility) dựa trên **Information Expert** (ai giữ thông tin thì làm).
 - Xác định "Cộng tác viên" (Collaborators) - những lớp cần tương tác để hoàn thành trách nhiệm.
 - *Ví dụ:* Lớp Hire cần tính tiền phạt, nhưng nó không biết giá thuê của xe → nó cần collaborate với lớp Bike.

Bước 4: Thiết kế Hệ thống

Đây là giai đoạn chuyển từ bài toán sang giải pháp phần mềm.

1. Kiến trúc phân tầng (Layering):

- **Presentation Layer (Boundary):** Hiển thị, nhận tương tác.
- **Application Layer (Control):** Điều phối use case, xử lý logic nghiệp vụ.
- **Domain Layer (Entity):** Logic cốt lõi, dữ liệu.
- **Infrastructure Layer (Persistence):** CSDL, kết nối.

2. Áp dụng BCE (Boundary - Control - Entity):

- Tách biệt logic hiển thị, logic điều phối và dữ liệu.
- *Quy tắc tương tác:* Tác nhân → Boundary → Control → Entity. Entity không được gọi ngược lên Boundary/Control.

3. Thiết kế chi tiết:

- **Sequence Diagram:** Nên sử dụng mô hình BCE. Có thể áp dụng thêm các khung alt, opt, loop, ref. Cần thể hiện rõ thông điệp (message) và đối tượng nhận.
- **Design Class Diagram (DCD):** Bổ sung phương thức, kiểu dữ liệu, tính điều hướng (Navigability). Đây là sơ đồ "chốt" để code.
- **Biểu đồ trạng thái (State Machine Diagram):** Dùng cho các đối tượng có vòng đời phức tạp (ví dụ: Order: Chờ thanh toán → Đã thanh toán → Đang giao → Đã nhận → Hoàn trả).

4. Một số nguyên tắc thiết kế:

- **Information Expert:** Tại sao gán trách nhiệm này cho lớp X? (Vì X có đủ dữ liệu).
- **Creator:** Tại sao lớp X tạo ra lớp Y? (Vì X chứa hoặc quản lý Y).
- **Controller:** Tại sao chọn lớp này làm Controller? (Vì nó đại diện cho hệ thống hoặc use case).
- **Low Coupling / High Cohesion:** Tại sao tách lớp này ra? (Để giảm phụ thuộc, tăng tính tập trung).
- **SOLID:** Chỉ ra các lớp tuân thủ SRP, OCP, DIP, ...

Bước 5: Thiết kế Cơ sở dữ liệu

1. Chuyển đổi từ Class Diagram sang ERD:

- Class → Table.
- Attribute → Column.
- Association 1-n → Thêm khóa ngoại vào bảng con (đầu 'n').
- Association n-m → Tạo bảng trung gian.
- Inheritance → Chọn chiến lược (Single Table hoặc Concrete Table).

2. Viết Script SQL: Tạo bảng, khóa chính, khóa ngoại, ràng buộc (Check, Unique).

3. Dữ liệu mẫu (Seed Data): Tạo sẵn dữ liệu để kiểm thử.

Bước 6: Cài đặt và Kiểm thử (Chương 4 – khuyến khích, không bắt buộc)

1. **Mapping UML to Code:** Chuyển đổi DCD thành code. Lưu ý: Các thuộc tính private, methods public/protected phải khớp với sơ đồ.
2. **Handling Errors:** Áp dụng nguyên lý SOLID để code dễ bảo trì (đặc biệt là Dependency Inversion - dùng Interface/Abstract class để giảm coupling).
3. **Kiểm thử:** Viết các Test Case cho các luồng chính và luồng ngoại lệ. Ghi lại các kết quả thực tế.

III. HƯỚNG DẪN ÁP DỤNG CÁC NGUYÊN LÝ THIẾT KẾ

1. Nguyên lý GRASP:

- **Information Expert:** "Ai có đủ dữ liệu để trả lời câu hỏi này?". Đừng bắt lớp Controller đi hỏi dữ liệu của Entity rồi tính toán. Hãy để Entity tự tính toán.
 - Ví dụ: Loan có borrowDate và period → Loan tự tính calculateDueDate().
- **Creator:** Ai sở hữu lớp đó thì lớp đó tạo ra nó.
 - Ví dụ: Sale chứa SalesLineItem → Sale có trách nhiệm tạo SalesLineItem.
- **Controller:** Giao diện (Boundary) không được chứa logic nghiệp vụ. Chỉ gọi phương thức của lớp Control.
 - Ví dụ: BorrowForm gọi LoanController.borrowBook().
- **High Cohesion:** Lớp chỉ nên làm một nhóm việc liên quan đến nhau. Nếu lớp Customer vừa quản lý thông tin, vừa in hóa đơn, vừa gửi email → Vi phạm (Cần tách lớp).

- **Low Coupling:** Giảm sự phụ thuộc bằng cách dùng Interface hoặc thông qua lớp trung gian.

2. Nguyên lý SOLID

- **SRP (Single Responsibility):** Mỗi lớp chỉ nên thay đổi vì một lý do duy nhất.
- **OCP (Open/Closed):** Mở rộng tính năng mới mà không phải sửa code cũ (dùng đa hình/interface).
- **LSP (Liskov Substitution):** Lớp con phải thay thế được lớp cha mà không làm hỏng hệ thống.
- **ISP (Interface Segregation):** Tránh các interface quá lớn.
- **DIP (Dependency Inversion):** Lớp cấp cao không phụ thuộc lớp cấp thấp, cả hai cùng phụ thuộc abstraction.

IV. CẤU TRÚC BÁO CÁO

Báo cáo được trình bày theo trình tự tiến hóa của hệ thống. Riêng Chương 4 (Cài đặt và Kiểm thử) không bắt buộc nhưng khuyến khích sinh viên thực hiện.

Chương 1: Phân tích yêu cầu

- Giới thiệu bài toán.
- Liệt kê Use Case (High-level & Expanded format).
- Biểu đồ Use Case (kèm quan hệ Include/Extend).
- Mô tả chi tiết các tác nhân (Actors).

Chương 2: Phân tích đối tượng

- Mô hình khái niệm (Conceptual Model).
- Thẻ CRC (Bảng hoặc file ảnh).
- Biểu đồ tuần tự hệ thống (System Sequence Diagram - SSD).

Chương 3: Thiết kế

- Kiến trúc tầng (Layering diagram).
- Sơ đồ các lớp BCE.
- Sequence Diagram (Cho 2-3 ca sử dụng phức tạp).
- Design Class Diagram (DCD): Phải có đầy đủ phương thức và thuộc tính.
- Biểu đồ trạng thái (Statechart) cho các thực thể vòng đời phức tạp.

- Phân tích GRASP/SOLID: Mỗi lớp quan trọng nên có giải thích: "Tại sao chọn Controller này? Tại sao lớp này lại có High Cohesion?"...

Chương 4: Cài đặt và Kiểm thử

- Cấu trúc project (Folder tree).
- Ảnh chụp màn hình (UI) tương ứng với Use Case.
- Kết quả Test Case.

V. GỢI Ý ĐỀ TÀI VÀ PHẠM VI YÊU CẦU

Dưới đây là các đề tài gợi ý, kèm **phạm vi yêu cầu cụ thể** để một nhóm 3-4 sinh viên năm cuối có thể hoàn thành trong **2-3 tuần**.

Các nhóm có thể chọn đúng phạm vi này, hoặc mở rộng/thu hẹp tùy theo năng lực.

1. Quản lý Thư viện

- **Phạm vi:** Quản lý sách (đầu sách, bản sao), độc giả, phiếu mượn/trả. Xử lý mượn sách, trả sách, tính phí phạt trễ hạn. Tìm kiếm sách theo tên/tác giả. Xử lý đặt trước sách khi sách đã được mượn hết. Quản lý trạng thái bản sao (Available, Loaned, Reserved, Lost, Damaged).
- **Yêu cầu tối thiểu:** 5 bảng (Sách, Bản sao, Độc giả, Phiếu mượn, Đặt trước). Có xử lý ngoại lệ (sách đã mượn, độc giả quá hạn, sách bị mất, đặt trước khi đã có người đặt).

2. Quản lý Sinh viên

- **Phạm vi:** Quản lý thông tin sinh viên, lớp học, môn học, điểm số. Nhập điểm, xem bảng điểm, thống kê điểm trung bình. Xử lý sinh viên học lại, học cải thiện. Phân quyền (Admin/Giảng viên/Sinh viên). Tính điểm theo hệ 10, hệ 4, tín chỉ.
- **Yêu cầu tối thiểu:** 5 bảng (Sinh viên, Lớp, Môn học, Điểm, Đăng ký môn). Phân quyền rõ ràng. Xử lý ngoại lệ (nhập điểm ngoài khoảng 0-10, trùng môn học, sinh viên chưa đăng ký môn).

3. Quản lý Khách sạn

- **Phạm vi:** Quản lý phòng, loại phòng, khách hàng, đặt phòng, trả phòng. Tính tiền phòng, dịch vụ. Kiểm tra phòng trống. Xử lý đặt phòng theo mùa, giá phòng thay đổi. Quản lý trạng thái phòng (Trống, Đã đặt, Đang ở, Dọn dẹp, Bảo trì).
- **Yêu cầu tối thiểu:** 5 bảng (Phòng, Loại phòng, Khách hàng, Đặt phòng, Hóa đơn). Xử lý ngoại lệ (đặt phòng trùng, hủy phòng, phòng đang bảo trì).

4. Quản lý Phòng Gym

- **Phạm vi:** Quản lý hội viên, gói tập, lớp học, huấn luyện viên. Đăng ký lớp học, gia hạn gói tập. Xử lý hội viên VIP, ưu đãi giảm giá. Kiểm tra sức chứa lớp học. Quản lý trạng thái hội viên (Active, Expired, Suspended).
- **Yêu cầu tối thiểu:** 5 bảng (Hội viên, Gói tập, Lớp học, Đăng ký, Huấn luyện viên). Xử lý ngoại lệ (lớp đã đầy, hội viên hết hạn, trùng lịch học).

5. Quản lý Phòng khám

- **Phạm vi:** Quản lý bệnh nhân, bác sĩ, lịch hẹn, hồ sơ bệnh án. Đặt lịch, khám bệnh, kê đơn thuốc. Xử lý bệnh nhân cấp cứu (ưu tiên), kiểm tra tương tác thuốc đơn giản. Quản lý trạng thái lịch hẹn (Chờ xác nhận, Đã xác nhận, Đã khám, Hủy).
- **Yêu cầu tối thiểu:** 5 bảng (Bệnh nhân, Bác sĩ, Lịch hẹn, Hồ sơ bệnh án, Đơn thuốc). Xử lý ngoại lệ (trùng lịch hẹn, bác sĩ không có lịch, bệnh nhân chưa có hồ sơ).

6. Hệ thống Bán hàng POS (Point of Sale)

- **Phạm vi:** Quản lý sản phẩm, khách hàng, hóa đơn, thanh toán. Xử lý bán hàng, in hóa đơn, tính tiền thừa. Xử lý nhiều phương thức thanh toán (Tiền mặt, Thẻ, Ví điện tử), áp dụng mã giảm giá. Quản lý trạng thái hóa đơn (Mới, Đã thanh toán, Đã hủy, Hoàn tiền).
- **Yêu cầu tối thiểu:** 6 bảng (Sản phẩm, Khách hàng, Hóa đơn, Chi tiết hóa đơn, Thanh toán, Khuyến mãi). Xử lý ngoại lệ (sản phẩm hết hàng, thanh toán thiếu, mã giảm giá hết hạn).

7. Quản lý Ký túc xá

- **Phạm vi:** Quản lý phòng, sinh viên, hợp đồng thuê phòng, hóa đơn tiền điện/nước. Xếp phòng, trả phòng. Xử lý chuyển phòng, gia hạn hợp đồng. Tính tiền điện/nước theo bậc thang. Quản lý trạng thái phòng (Trống, Đã thuê, Bảo trì).
- **Yêu cầu tối thiểu:** 5 bảng (Phòng, Sinh viên, Hợp đồng, Hóa đơn, Chỉ số điện nước). Xử lý ngoại lệ (phòng đã đầy, hợp đồng hết hạn, chỉ số điện nước không hợp lệ).

8. Quản lý Cửa hàng Điện thoại

- **Phạm vi:** Quản lý sản phẩm (điện thoại), nhà cung cấp, khách hàng, hóa đơn nhập/xuất. Quản lý tồn kho, bán hàng. Cảnh báo tồn kho thấp, quản lý bảo hành. Quản lý trạng thái bảo hành (Tiếp nhận, Đang sửa, Đã trả, Từ chối).
- **Yêu cầu tối thiểu:** 6 bảng (Sản phẩm, Nhà cung cấp, Khách hàng, Hóa đơn nhập, Hóa đơn xuất, Bảo hành). Xử lý ngoại lệ (sản phẩm hết hàng, bảo hành hết hạn, nhập sai số lượng).

9. Quản lý Công việc (Task Management)

- **Phạm vi:** Quản lý dự án, công việc, nhân viên. Giao việc, cập nhật trạng thái công việc, thống kê tiến độ. Xử lý dependency giữa các công việc, phân công dựa trên kỹ năng. Quản lý trạng thái công việc (Mới, Đang làm, Hoàn thành, Tạm dừng).
- **Yêu cầu tối thiểu:** 5 bảng (Dự án, Công việc, Nhân viên, Phân công, Kỹ năng). Xử lý ngoại lệ (công việc phụ thuộc chưa xong, nhân viên quá tải, trùng lịch).

10. Quản lý Nhà hàng (Restaurant Management)

- **Phạm vi:** Quản lý bàn, thực đơn, order, thanh toán. Xử lý đặt bàn, gọi món, chuyển bàn, tách hóa đơn. Quản lý trạng thái bàn (Trống, Đã đặt, Đang phục vụ, Dọn dẹp). Tính tiền theo món, áp dụng giảm giá, phí phục vụ.
- **Yêu cầu tối thiểu:** 6 bảng (Bàn, Thực đơn, Order, Chi tiết Order, Hóa đơn, Khuyến mãi). Xử lý ngoại lệ (bàn đã có khách, món hết, order trùng, thanh toán thiếu).

VI. LƯU Ý:

1. Các nhóm có thể chọn một trong các đề tài trên, hoặc đề xuất một đề tài khác với phạm vi tương đương.
2. Yêu cầu tối thiểu về số bảng chỉ là gợi ý; nhóm có thể thiết kế ít hơn nếu gộp bảng hợp lý, nhưng phải đảm bảo chuẩn hóa.
3. Các nhóm có thể sử dụng AI để hỗ trợ viết code mẫu, nhưng phải tự kiểm chứng và điều chỉnh thiết kế.
4. Dùng công cụ: Tuyệt đối không vẽ tay. Hãy dùng StarUML, Visual Paradigm, Mermaid... để quản lý mô hình. Lưu file thiết kế để dễ chỉnh sửa.
5. Tránh "Over-engineering": Tránh cố tạo quá nhiều lớp vô nghĩa. Một lớp tốt phải phản ánh đúng bài toán (Problem domain). Nếu một lớp chỉ có 1-2 phương thức đơn giản, hãy cân nhắc gộp vào lớp khác.